Celo-Monorepo Audit Report

Executive Summary

Scope

Disclaimer

Auditing Process

Vulnerability Severity

Findings

- [High] Incomplete delegation balance updates in slash and unlock function…
- [Low] Potential division-by-zero risk in _getSellTokenAmount function lea…
- [Low] EpochRewards._getTargetGoldSupplyIncrease will be DoS due to divisi…
- [Low] historySize permanently exceeds retention window in Random.sol
- [Info] Incorrect array length deletion

Executive Summary

On May 27,2025, the celo-org team engaged Fuzzland to conduct an automated security audit of their project. The primary objective was identifying and mitigating potential security vulnerabilities, risks, and coding issues to enhance the project's robustness and reliability.

Scope

Project Name	Celo-org
Repo	celo-monorepo
Commit	`6d6f9a49cfb601a397313c3c8d344ef0302eafcb`
Fixed	
Language	Solidity
Scope	celo-monorepo/packages/protocol/contracts/*.sol

Disclaimer

The audit does not ensure that it has identified every security issue in the smart contracts, and it should not be seen as a confirmation that there are no more vulnerabilities. The audit is not exhaustive, and we recommend further independent audits and setting up a public bug bounty program for enhanced security verification of the smart contracts. Additionally, this report should not be interpreted as personal financial advice or recommendations.

Auditing Process

- Static Analysis: We perform static analysis using our internal tools and Slither to identify potential vulnerabilities and coding issues.
- Fuzz Testing: We execute fuzz testing with our internal fuzzers to uncover potential bugs and logic flaws.
- Invariant Development: We convert the project into Foundry project and develop Foundry invariant tests for the project based on the code semantics and documentations.
- Invariant Testing: We run multiple fuzz testing tools, including Foundry and ItyFuzz, to identify violations of invariants we developed.
- Formal Verification: We develop individual tests for critical functions and leverage Halmos to prove the functions in question are not vulnerable.

Vulnerability Severity

We divide severity into three distinct levels: high, medium, low. This classification helps prioritize the issues identified during the audit based on their potential impact and urgency.

- **High Severity Issues** represent critical vulnerabilities or flaws that pose a significant risk to the system's security, functionality, or performance. These issues can lead to severe consequences such as fund loss, or major service disruptions if not addressed immediately. High severity issues typically require urgent attention and prompt remediation to mitigate potential damage and ensure the system's integrity and reliability.
- **Medium Severity Issues** are significant but not critical vulnerabilities or flaws that can impact the system's security, functionality, or performance. These issues might not pose an immediate threat but have the potential to cause considerable harm if left unaddressed over time. Addressing medium severity issues is important to maintain the overall health and efficiency of the system, though they do not require the same level of urgency as high severity issues.
- **Low Severity Issues** are minor vulnerabilities or flaws that have a limited impact on the system's security, functionality, or performance. These issues generally do not pose a significant risk and can be addressed in the regular maintenance cycle. While low severity issues are not critical, resolving them can help improve the system's overall quality and user experience by preventing the accumulation of minor problems over time.

Below is a summary of the vulnerabilities with their current status, highlighting the number of issues identified in each severity category and their resolution progress.

	Number	Resolved
High Severity Issues	1	0
Medium Severity Issues	0	0
Low Severity Issues	3	0
Info Severity Issues	1	0

Findings

[High] Incomplete delegation balance updates in slash and unlock functions of lockedGold contract

In the `LockedGold` contract, the `slash` and `unlock` functions do not fully update the delegation balance information of related accounts when handling account balance changes, which may lead to inaccurate delegation balance calculations.

In the `slash` function, when an account is penalized and the reporter receives a reward, the code only updates the delegation information of the penalized account but not the reporter's delegation information:

```
function slash( address account, uint256 penalty, address reporter, uint256
reward, address[] calldata lessers, address[] calldata greaters, uint256[]
calldata indices ) external onlySlasher onlyWhenNotBlocked { // ... other code
... _incrementNonvotingAccountBalance(reporter, reward);
_updateDelegatedAmount(account); // Missing delegation balance update for
reporter: _updateDelegatedAmount(reporter); // ... other code ... }
```

When the `reporter` receives locked CELO as a reward, their non-voting balance increases, but the delegation balance is not updated accordingly, which may lead to inaccurate delegation ratio calculations.

In the `unlock` function, although `revokeFromDelegatedWhenUnlocking` is called to handle the delegator's delegation revocation, the account's status as a delegatee is not updated:

```
function unlock(uint256 value) external nonReentrant { // ... other code ...
FixidityLib.Fraction memory delegatedPercentage = delegatorInfo[msg.sender]
.totalDelegatedCeloFraction; if (FixidityLib.gt(delegatedPercentage,
FixidityLib.newFixed(0))) { revokeFromDelegatedWhenUnlocking(msg.sender,
value); } _decrementNonvotingAccountBalance(msg.sender, value); // Missing
update for msg.sender's status as a delegatee // Should add:
_updateDelegatedAmount(msg.sender); // ... other code ... }
```

When an account unlocks CELO, if the account is also a delegatee for other accounts, its status as a delegatee is not updated, which may lead to inaccurate delegation balance calculations.

Recommendation:

In the `slash` function, add an update to the reporter's delegation balance after increasing their non-voting balance:

```
function slash( // ... parameters ... ) external onlySlasher
onlyWhenNotBlocked { // ... other code ...
_incrementNonvotingAccountBalance(reporter, reward);
_updateDelegatedAmount(account); if (reporter != address(0)) {
_updateDelegatedAmount(reporter); // Add this line } // ... other code ... }
```

Before the end of the `unlock` function, add an update to the account's status as a delegatee:

```
function unlock(uint256 value) external nonReentrant { // ... other code ...
_decrementNonvotingAccountBalance(msg.sender, value);
_updateDelegatedAmount(msg.sender); // Add this line // ... other code ... }
```

[Low] Potential division-by-zero risk in `_getSellTokenAmount` function leading to Denial of Service (DoS)

The `_getSellTokenAmount` function within the CompileExchange.sol contract is susceptible to a division-by-zero error when calculating the amount of tokens to sell if the `buyAmount` (amount of tokens to buy) is equal to the `buyTokenBucket` (reserve bucket size for the buy token). This will cause the relevant transaction to revert, creating a Denial of Service (DoS) scenario.

Recommendation:

Before performing the division, add a check to ensure that `buyAmount` is strictly less than buyTokenBucket .It is recommended to add the following check at the beginning of the `_getSellTokenAmount` function.

[Low] `EpochRewards._getTargetGoldSupplyIncrease` will be DoS due to division by 0

When `communityRewardFraction + carbonOffsettingFraction = 1`, `_getTargetGoldSupplyIncrease` will be reverted due to division by 0, causing `calculateTargetEpochRewards` to be DoS, causing `EpochManager.startNextEpochProcess` to revert, resulting in rewards not being distributed normally.

Recommendation:

When setting `communityRewardFraction` and `carbonOffsettingFraction`, make sure their sum is not 1.

[Low] `historySize` permanently exceeds retention window in `Random`.sol

When `historySize > deprecated_randomnessBlockRetentionWindow`, `addRandomness` deletes two history entries but only decrements `historySize` by 1. As multiple transactions accumulate, `historySize > deprecated_randomnessBlockRetentionWindow` may be true for a long time.

Recommendation:

When `if (historySize > deprecated_randomnessBlockRetentionWindow)`, `historySize = historySize.sub(2)` instead of `historySize = historySize.sub(1)` .

[Info] Incorrect array length deletion

`Governance.dequeueProposalsIfReady` chooses to modify the array via `emptyIndices.length = indexOfLastEmptyIndex`, which does not correctly delete elements from the array, but only reduces the length of the array without deleting the memory of the "deleted" elements, leaving the array with accessible stale data.

```
function dequeueProposalsIfReady() public { //...... if (emptyIndices.length
!= 0) { uint256 indexOfLastEmptyIndex = emptyIndices.length.sub(1);
dequeued[emptyIndices[indexOfLastEmptyIndex]] = proposalId; delete
emptyIndices[indexOfLastEmptyIndex]; emptyIndices.length =
indexOfLastEmptyIndex; //......
```

Recommendation:
Use `pop()` instead of manual length manipulation.